Frontend Roadmap for 50 LPA

Sach ye hai: React toh sabko aata hai. 50 LPA milta hai React ke **aas-paas** ki cheezon se - rendering, performance, security, architecture. Isliye is sheet me React sabse **last** me hai, pehle nahi. Order maintain karo.
Open in Desktop view for Better Visibility.
Feel Free to DM @kaash.tech for doubts or query.

What's Covered (At a Glance)

Level	Focus	Kaunse topics	Yahan tum ho toh...
1. Basics	HTML / CSS / JS foundation	Semantic HTML, CSS layout + units, JS core, DOM, Git	Fresher / 0-1 saal - pakka karo, yahan atakna nahi
2. Intermediate	JS deep + React core + APIs	Closures/async, hooks, REST, state mgmt, TypeScript	~6-15 LPA zone - yahan zyada log ruk jaate hain
3. Expert	Rendering, perf, security, architecture	SSR/hydration, diffing, bundling/tree shaking, GraphQL/auth, Next.js, testing, FE system design	30-50 LPA zone - yahi tumhe alag banata hai

Rule: Level 1 ke 90% aaye bina Level 2 mat shuru karo. React (Level 2 me) tab tak mat chhuo jab tak JS core solid na ho.

How To Use This Sheet

1. Upar se neeche chalo - level skip mat karo, foundation weak reh jayega.
2. Har topic: concept samjho → chhota demo khud banao → ek real project me use karo. 3. "Padha hai" nahi, "bana hai" - har Expert topic ka ek working example hona chahiye portfolio me. 4. Interview me har cheez ka **"kyun"** bolo, sirf "kya" nahi. 50 LPA "kyun" samajhne walon ko milta hai.

Level 1 - Basics (Foundation, sabko aana chahiye)

1. HTML

1. Semantic tags (header, nav, main, section, article, footer) - SEO + accessibility dono.
2. Forms + validation (input types, labels, required, pattern).
3. Accessibility basics (alt text, aria-label, keyboard navigation, heading order).
4. Meta tags + basic SEO (title, description, viewport, Open Graph).

2. CSS

1. Box model (content, padding, border, margin) - poora layout isi pe khada hai.
2. Flexbox (main axis, cross axis, justify/align) - 1D layouts.
3. Grid (rows, columns, areas) - 2D layouts.
4. **Units - kaha kya use karna hai** (ye alag hi differentiator hai):
    i. `rem` → text + spacing (root 16px se scale)
    ii. `em` → padding (element ke font-size se, par nested me compound hota hai)
    iii. `px` → border + shadow (fixed cheezein)
    iv. `%` → width/height (parent ke hisaab se)
    v. `dvh` → full screen (mobile 100vh address-bar bug se bachne ke liye)
5. Positioning (relative, absolute, fixed, sticky) + z-index.
6. Responsive design - media queries, mobile-first, `min()/max()/clamp()` .

3. JavaScript Core

1. `var / let / const` , scope, hoisting.
2. Data types, type coercion, `==` vs `===` .
3. Functions, arrow functions, default + rest params.
4. Array methods (map, filter, reduce, find, some, every).
5. Object methods, destructuring, spread/rest.
6. DOM manipulation (querySelector, createElement, classList).
7. Events (addEventListener, event object, bubbling vs capturing basics).

4. Tooling Basics

1. Git + GitHub (add, commit, branch, merge, pull request).
2. npm/pnpm basics (install, scripts, package.json).
3. Browser DevTools (Elements, Console, Network tab padhna).

Level 2 - Intermediate (Yahan se log alag hote hain)

1. JavaScript Deep Dive

1. Closures (aur unka real use - data privacy, memoization).
2. `this` binding (call, apply, bind, arrow-function ka `this`).
3. Prototypes + prototypal inheritance.
4. **Event loop** - call stack, callback queue, microtask queue (interview classic).
5. Callbacks → Promises → async/await → error handling (try/catch).
6. ES6+ modules (import/export), optional chaining, nullish coalescing.

2. React Core

1. Components (function components), JSX, props vs state.
2. Hooks: `useState`, `useEffect` (dependency array + cleanup), `useRef`. 3. `useMemo`, `useCallback`, `useContext` - kab use karna hai (aur kab nahi). 4. Conditional rendering, lists + keys (keys kyun zaroori hain).
5. Forms - controlled vs uncontrolled components.
6. Lifting state up, prop drilling ka problem.
7. Component lifecycle (mount, update, unmount) via hooks.

3. Networking + Data

1. REST APIs - HTTP methods (GET/POST/PUT/PATCH/DELETE), status codes.
2. `fetch` / `axios`, request/response handling, loading + error states.
3. JSON, CORS kya hai aur kyun blocker banta hai.
4. Async data fetching patterns in React (useEffect + fetch, ya React Query intro).

4. State Management

1. Context API (kab kaafi hai).
2. Redux Toolkit ya Zustand basics (global state kab chahiye).
3. Server state vs client state ka fark.

5. CSS + TypeScript

1. Transitions + animations (transform, transition, keyframes).
2. Tailwind CSS ya CSS-in-JS (styled-components).
3. TypeScript basics - types, interfaces, generics, props typing in React.

Level 3 - Expert (50 LPA - yahi cheezein differentiate karti hain)

1. Rendering + Performance

1. CSR vs SSR vs SSG vs ISR - kaunsa kab (SEO, TTFB, dynamic data).
2. **Hydration** - SSR ke baad JS attach hona, aur "hydration mismatch" bug. 3. **React reconciliation / diffing algorithm** - virtual DOM kaise compare hota hai, keys ka role. 4. Memoization at scale - `React.memo` , `useMemo` , `useCallback` (aur inka over-use ka nuksaan).
5. Code splitting + lazy loading (`React.lazy` , `Suspense` , dynamic import).
6. **Core Web Vitals** - LCP, CLS, INP (pehle FID). Inhe measure aur improve karna.
7. Image optimization (lazy load, srcset, next/image), font optimization.

2. Build Tools + Architecture

1. Bundlers - Webpack vs Vite vs esbuild (kaam kya karte hain).
2. **Tree shaking** - dead code kaise hata hai bundle se.
3. **Code splitting** - bundle ko chunks me todna.
4. Design systems + reusable component libraries.
5. Component architecture - composition, container/presentational, folder structure.
6. Monorepos (Turborepo/Nx) + micro-frontends (bade orgs me).

3. Networking + Security

1. GraphQL - queries, mutations, kab REST se better.
2. WebSockets - real-time (chat, live updates).
3. **Authentication** - JWT, OAuth, sessions vs tokens, cookies (httpOnly, SameSite).
4. Web security - XSS, CSRF, CSP, sanitization.
5. Caching - HTTP cache headers, CDN, service workers, PWA basics.

4. Advanced React + Framework

1. Custom hooks (reusable logic).
2. Error boundaries, portals, refs forwarding.
3. Suspense + concurrent features (transitions, streaming).
4. **Next.js** - App Router, Server Components, data fetching, SSR/ISR, route handlers.
5. React Query / TanStack Query - server state, caching, invalidation.

5. Testing

1. Unit testing - Jest / Vitest.
2. Component testing - React Testing Library (behaviour test karo, implementation nahi).
3. E2E testing - Playwright / Cypress.

6. Frontend System Design (senior/50 LPA rounds)

1. Component + state architecture bade app me.
2. API design + data flow (client-server contract).
3. Performance budgets, accessibility at scale.
4. Trade-offs bolna - SSR vs CSR, monolith vs micro-frontend, kyun.

Important Notes

1. **React last me kyun?** Kyunki React sabko aata hai - wahi tumhe average banata hai. Paisa uske aas-paas ki depth (rendering, perf, security, architecture) me hai.
2. **Ratna nahi, banana hai.** Har Expert topic ka ek chhota project ya demo banao - interview me "maine use kiya hai" bolna "maine padha hai" se 10x strong hai.
3. **Depth > breadth.** 20 tools surface-level jaanne se acha hai 5 cheezein deeply samajhna aur "kyun" bolna.
4. **Level 1-2 solid without shortcut.** Expert topics tabhi samajh aayenge jab JS core + React core pakka ho.
5. **50 LPA = problem solver, not feature builder.** Trade-offs, performance, aur system-level sochna - yahi differentiate karta hai.

Beyond the Degree - React se aage niklo. Jo React ke aas-paas aata hai, wahi tumhe 50 LPA tak le jaata hai. See you in the next one.